

W14D03 — Alembic: Database Migration

ML Engineer Journey | Phase 4 — Database & Observability | 10 Jun 2026

1. Analogi — Git untuk Schema Database

Bayangkan schema database seperti kodebase. Setiap perubahan — tambah kolom, ubah tipe, hapus tabel — adalah sebuah *perubahan* yang perlu di-track. Tanpa tracking, tidak ada history, tidak ada rollback, tidak ada cara tahu apa yang berubah dan kapan. **Alembic adalah Git-nya schema database**. Setiap perubahan schema menjadi sebuah *migration file* — script Python yang tahu cara *upgrade* (apply perubahan) dan *downgrade* (batalkan perubahan). File ini bisa di-commit ke Git, di-review, dan dijalankan ulang di environment manapun.

2. Cara Kerja Alembic

Alembic bekerja dengan membandingkan dua hal: **model SQLAlchemy** (source of truth — apa yang kamu *inginkan*) dengan **database aktual** (kondisi yang *sekarang ada*). Hasil perbandingan ini di-generate menjadi migration script. Alembic menyimpan state migration di tabel khusus bernama `alembic_version` langsung di dalam database — bukan di file lokal. Database sendiri yang tahu sudah di revision mana.

Langkah	Perintah	Yang Terjadi
Detect diff	<code>alembic revision --autogenerate -m '...'</code>	Alembic bandingkan Base.metadata vs DB, generate .py file
Apply ke DB	<code>alembic upgrade head</code>	Jalankan <code>upgrade()</code> di migration file, update <code>alembic_version</code>
Cek posisi	<code>alembic current</code>	Tampilkan revision ID yang aktif di DB saat ini
Lihat history	<code>alembic history --verbose</code>	Tampilkan seluruh rantai migration dengan Parent-nya
Rollback 1 step	<code>alembic downgrade -1</code>	Jalankan <code>downgrade()</code> dari revision terakhir
Rollback spesifik	<code>alembic downgrade <rev_id></code>	Rollback ke revision tertentu
Reset total	<code>alembic downgrade base</code>	Jalankan semua <code>downgrade()</code> — DB kembali kosong

3. Setup Alembic

Install dan inisialisasi

```
pip install alembic
alembic init alembic
```

Struktur yang di-generate:

```
alembic/
├── env.py           ← konfigurasi utama
├── script.py.mako   ← template migration file
├── versions/        ← semua migration file tersimpan di sini
└── alembic.ini      ← konfigurasi database URL
```

Konfigurasi alembic.ini

```
sqlalchemy.url = postgresql+psycopg2://user:password@localhost:5432/dbname
```

Best practice production: Jangan hardcode credentials di alembic.ini. Baca dari environment variable di env.py menggunakan python-dotenv, lalu set config.set_main_option('sqlalchemy.url', os.getenv('DATABASE_URL')).

Konfigurasi env.py — bagian paling kritis

```
import sys, os
sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

from week_14.models import Base    # import Base dari models kamu

target_metadata = Base.metadata    # INI yang enable autogenerate
```

Tanpa target_metadata = Base.metadata, Alembic tidak tahu struktur yang kamu inginkan — ia hanya bisa generate migration kosong. Dengan ini, Alembic scan semua class yang inherit Base dan bandingkan dengan state DB saat ini.

4. Anatomy Migration File

```
# alembic/versions/abc123_initial_schema.py

revision = 'abc123'    # ID unik migration ini
down_revision = None    # None = migration pertama, tidak ada parent

def upgrade() -> None:
    op.create_table('users',
        sa.Column('id', postgresql.UUID(as_uuid=True), nullable=False),
        sa.Column('username', sa.String(100), nullable=False),
        sa.PrimaryKeyConstraint('id')
    )

def downgrade() -> None:
    op.drop_table('users')    # urutan terbalik karena foreign key
```

down_revision membentuk rantai linear. Alembic membaca chain ini untuk tahu urutan eksekusi. Saat kamu tambah migration baru, down_revision-nya otomatis di-set ke revision sebelumnya. Ini yang memungkinkan upgrade dan downgrade berjalan dalam urutan yang benar.

Contoh rantai tiga migration:

```
<base>  →  5cf9b410db15 (initial schema)
        →  db5b9731e1d4 (add efficiency_index)
        →  24f6544a1461 (add model_version)  ← HEAD
```

5. Autogenerate vs Manual Migration

	--autogenerate	Manual
Cara buat	alembic revision --autogenerate -m '...'	alembic revision -m '...'
Isi script	Di-generate Alembic dari diff model vs DB	Kamu tulis sendiri op.add_column() dll
Cocok untuk	Tambah/hapus tabel, kolom, index	Migrasi data, rename kolom, custom SQL
Detect rename?	TIDAK — dianggap drop + add baru	Ya, tulis op.alter_column() manual
Detect data change?	TIDAK — hanya schema	Ya, tulis manual dengan UPDATE SQL
Wajib di-review?	Ya, selalu baca sebelum upgrade	Ya, human error lebih tinggi

Peringatan: Autogenerate tidak 100% sempurna. Yang sering tidak terdeteksi: rename tabel/kolom (dianggap drop + create = DATA LOSS), perubahan stored procedure, perubahan data. Selalu buka dan baca file yang di-generate sebelum menjalankan alembic upgrade.

6. alter_column — Bukan rename_column

Ini salah satu yang paling sering bikin bingung. Di Alembic **tidak ada** `op.rename_column()`. Rename kolom dilakukan lewat `op.alter_column()` — sama seperti SQL ALTER COLUMN yang bisa ubah banyak hal sekaligus.

```
# Rename kolom
op.alter_column('users', 'username', new_column_name='display_name')

# Ubah tipe
op.alter_column('users', 'age', type_=sa.String(10))

# Ubah nullable
op.alter_column('users', 'email', nullable=False)

# Kombinasi: rename + ubah tipe sekaligus
op.alter_column('users', 'username',
                new_column_name='display_name',
                type_=sa.String(200))
```

Kapan alter_column berbahaya untuk data?

Rename via `alter_column` aman — PostgreSQL hanya ubah nama di catalog, data tidak disentuh. Tapi **ubah tipe kolom yang sudah ada datanya bisa merusak data**:

```
# BERBAHAYA jika kolom sudah punya data
op.alter_column('predictions', 'risk_score',
                type_=sa.String(50)) # Float → String

# PostgreSQL coba cast setiap baris
# Cast gagal → migration error (seluruh operasi dibatalkan)
# Cast berhasil tapi lossy → data berubah diam-diam tanpa peringatan
```

Untuk ubah tipe kolom yang sudah ada datanya, selalu tulis migration manual dengan tiga langkah: (1) tambah kolom baru dengan tipe yang benar, (2) copy + transform data dari kolom lama, (3) drop kolom lama.

7. Merge Conflict Migration — alembic merge

Terjadi ketika dua orang membuat migration dari revision yang sama secara paralel. Hasilnya: dua migration file yang sama-sama menunjuk ke parent yang sama — Alembic tidak tahu mana yang harus dijalankan duluan.

```
# Situasi: branch conflict
# Orang A buat migration B (parent: A)
# Orang B buat migration C (parent: A)
# Hasil: A → B dan A → C — dua kepala

alembic heads # tampilkan semua 'kepala' yang ada
# Output: B (head), C (head)

# Resolve dengan merge
alembic merge -m 'merge branch conflict' B C
```

```
# Generate migration baru D dengan down_revision = (B, C)
```

```
alembic upgrade head # sekarang rantai linear kembali
```

Kenapa ini bisa terjadi? Tim besar yang bekerja di branch Git berbeda seringkali membuat migration secara paralel. Solusinya selalu komunikasikan siapa yang sedang buat migration, atau gunakan alembic merge untuk resolve sebelum merge ke main branch.

8. Ringkasan Perintah Penting

Perintah	Fungsi
alembic init alembic	Inisialisasi folder alembic/ di project
alembic revision --autogenerate -m '...'	Generate migration file dari diff model vs DB
alembic revision -m '...'	Generate migration file kosong (isi manual)
alembic upgrade head	Apply semua migration yang belum dijalankan
alembic upgrade +1	Maju satu step (untuk debug bertahap)
alembic downgrade -1	Rollback satu step
alembic downgrade base	Rollback ke state kosong
alembic current	Tampilkan revision aktif di DB saat ini
alembic history --verbose	Tampilkan seluruh rantai migration
alembic heads	Tampilkan semua 'kepala' (deteksi branch conflict)
alembic merge -m '...' <rev1> <rev2>	Gabungkan dua branch migration yang conflict

9. Checklist Sebelum alembic upgrade di Production

1. Baca seluruh isi migration file yang baru di-generate
2. Pastikan tidak ada op.drop_column atau op.drop_table yang tidak disengaja
3. Jika ada ubah tipe — pastikan tidak ada data yang akan corrupt
4. Jalankan alembic upgrade head di environment development dulu
5. Backup database sebelum jalankan di production
6. Pastikan downgrade() ditulis dengan benar — ini safety net-mu
7. Commit migration file ke Git sebelum apply ke production